

# การสร้างชุดข้อมูลข้ามแพลตฟอร์มสำหรับ การตรวจจับมัลแวร์ด้วยการเรียนรู้ของเครื่อง

อง

## จากบันทึกเหตุการณ์ Sysmon บนระบบปฏิบัติการ Linux และ Windows

### A CROSS-PLATFORM SYSMON-BASED DATASET FOR MACHINE-LEARNING MALWARE DETECTION ON LINUX AND WINDOWS

#### บทคัดย่อ

งานวิจัยด้านการตรวจจับมัลแวร์ด้วยการเรียนรู้ของเครื่องส่วนใหญ่อาศัยบันทึกเหตุการณ์จากระบบปฏิบัติการ Windows เพียงแพลตฟอร์มเดียว

บทความนี้นำเสนอการสร้างชุดข้อมูลสำหรับตรวจจับมัลแวร์จากบันทึกเหตุการณ์ Sysmon แบบสองแพลตฟอร์มคู่ขนานคือ Linux และ Windows โดยจำลองพฤติกรรมด้วยชุดทดสอบ Atomic Red Team ครอบคลุม 6 ประเภทสถานการณ์ ได้แก่ พฤติกรรมปกติ เรียกค่าไถ่ โทรจัน บอตเน็ต ชุดเหรียญ และการใช้ช่องโหว่ ชุดข้อมูลที่ได้มีทั้งสิ้น 81,331 เหตุการณ์ (Linux 45,311 และ Windows 36,020) มีสัดส่วนตัวอย่างอันตราย 33.8% ใกล้เคียงกันทั้งสองแพลตฟอร์ม

การกำหนดป้ายกำกับใช้การสืบสายกระบวนการจากไตรรกทอริททดสอบ ผลการทดลองพบว่าเมื่อประเมินด้วยการแบ่งข้อมูลแบบ GroupKFold ตามกระบวนการเพื่อป้องกันการรั่วไหลของข้อมูลแบบจำลอง Random Forest ระดับกระบวนการให้ความแม่นยำ 94.9% และค่า F1 เท่ากับ 0.924 ขณะที่แบบจำลองไม่มีผู้สอน Local Outlier Factor ให้พื้นที่ใต้เส้นโค้ง 0.849

บทความนี้ยังวิเคราะห์แหล่งการรั่วไหลของข้อมูลอย่างเป็นระบบและชี้ให้เห็นว่าโปรโตคอลการประเมินมีผลต่อค่าที่รายงานมากกว่าปริมาณข้อมูล

คำสำคัญ -- การตรวจจับมัลแวร์, Sysmon, การเรียนรู้ของเครื่อง, ชุดข้อมูลข้ามแพลตฟอร์ม, การรั่วไหลของข้อมูล

#### ABSTRACT

Most machine-learning malware-detection studies rely on host telemetry from a single operating system, typically Windows. This paper presents the construction of a malware-detection dataset from Sysmon event logs across two parallel platforms, Linux (Sysmon for Linux) and Windows, using Atomic Red Team to emulate six behavioral scenarios: benign, ransomware, trojan, botnet, cryptominer, and exploitation. The resulting dataset contains 81,331 events (45,311 on Linux and 36,020 on Windows) with a 33.8% malicious ratio that is nearly identical on both platforms. Labels are assigned by process lineage traced from a controlled sandbox directory. Under a leakage-controlled GroupKFold protocol (grouped by process), a process-level Random Forest attains 94.9% accuracy and an F1 of 0.924, while the unsupervised Local Outlier Factor reaches an AUC of 0.849. We further analyze the leakage sources that would otherwise inflate reported accuracy, and show that the evaluation protocol affects the reported figures more than the amount of data.

*Keywords -- malware detection, Sysmon, machine learning, cross-platform dataset, data leakage*

#### 1. บทนำ

การเปลี่ยนผ่านสู่ยุคดิจิทัลทำให้ภัยคุกคามจากมัลแวร์ทวีความรุนแรงและตรวจจับได้ยากขึ้น สถิติของหน่วยงานด้านความมั่นคงปลอดภัยไซเบอร์ในหลายประเทศชี้ว่าจำนวนเหตุการณ์โจมตีเพิ่มขึ้นต่อเนื่องทุกปี การเฝ้าระวังแบบอาศัยลายเซ็น (signature-based) เพียงอย่างเดียวจึงตามไม่ทันมัลแวร์รูปแบบใหม่

ทำให้การตรวจจับเชิงพฤติกรรมด้วยการเรียนรู้ของเครื่อง (machine learning) ได้รับความสนใจอย่างกว้างขวาง [1], [8]

เครื่องมือ System Monitor (Sysmon) เป็นแหล่งบันทึกเหตุการณ์ระดับโฮสต์ที่ให้อะเอียดของการสร้างกระบวนการ การเชื่อมต่อเครือข่าย และการเข้าถึงไฟล์อย่างครบถ้วน [20]

งานวิจัยอ้างอิงหลักของบทความนี้ [1] ได้สร้างชุดข้อมูลและเปรียบเทียบแบบจำลอง 7 ชนิดจากบันทึก Sysmon บนระบบ Windows และรายงานว่าการใช้แบบจำลอง Random Forest ให้ค่า F1 ที่ 0.8868 อย่างไรก็ตามงานดังกล่าวมีข้อจำกัดสองประการที่บทความนี้มุ่งแก้ไข ประการแรก ชุดข้อมูลครอบคลุมเฉพาะระบบปฏิบัติการ Windows

ทำให้ไม่สามารถประเมินได้ว่าคุณลักษณะที่ใช้ตรวจจับยังใช้ได้กับแพลตฟอร์มอื่นหรือไม่

ซึ่งมีหลักฐานว่าการโอนย้ายแบบจำลองข้ามระบบปฏิบัติการทำให้ค่า F1 ลดลงอย่างมีนัยสำคัญ [4] ประการที่สองงานดังกล่าวระบุเองว่าบันทึกที่ใช้ขาดความสัมพันธ์แบบกระบวนการแม่-ลูก (parent-child correlation) ซึ่งเป็นข้อมูลสำคัญต่อการวิเคราะห์เชิงพฤติกรรม [11]

บทความนี้มีส่วนสนับสนุน (contribution) สี่ประการ (1) สร้างชุดข้อมูล Sysmon แบบสองแพลตฟอร์มคู่ขนาน (Linux และ Windows) ภายใต้วิธีการเก็บและกำหนดป้ายกำกับเดียวกัน (2) เพิ่มคุณลักษณะความสัมพันธ์กระบวนการแม่-ลูกและบรรทัดคำสั่งทำงานอ้างอิงไม่มี (3) ครอบคลุมเหตุการณ์การเชื่อมต่อเครือข่าย (NetworkConnect) สำหรับพฤติกรรม C2/beaconing ซึ่งชุดข้อมูลอ้างอิงแทบไม่มี (เพียง 20 เหตุการณ์) และ (4) วิเคราะห์แหล่งการรั่วไหลของข้อมูล (data leakage) อย่างเป็นระบบตามข้อเสนอแนะเชิงระเบียบวิธีในงาน [2], [3] และแสดงให้เห็นว่าโปรโตคอลการประเมินมีผลต่อค่าความแม่นยำที่รายงานมากกว่าปริมาณข้อมูล

## 2. งานที่เกี่ยวข้อง

### 2.1 การตรวจจับมัลแวร์จากบันทึกเหตุการณ์และ Sysmon

การตรวจจับมัลแวร์เชิงพฤติกรรมจากบันทึกเหตุการณ์ระดับโฮสต์เป็นแนวทางที่ได้รับการศึกษาอย่างต่อเนื่อง งานอ้างอิงหลัก [1] เก็บบันทึก Sysmon จากเครื่อง 5 เครื่องในโดเมน Active Directory ต่อเนื่องประมาณ 31 ชั่วโมง และรวมศูนย์ผ่าน Windows Event Forwarding ได้ชุดข้อมูล NLME จำนวน 71,017 เหตุการณ์ จากนั้นเลือกคุณลักษณะด้วย Principal Component Analysis (PCA) และเปรียบเทียบแบบจำลองแบบมีผู้สอนและไม่มีผู้สอน งาน [5] เสนอระบบวิเคราะห์มัลแวร์แบบพลวัตด้วย Sysmon ร่วมกับ Elastic Stack และสร้างชุดข้อมูล AAU\_MalData จากการวิเคราะห์มัลแวร์ 2,800 ตัวอย่าง โดยเชื่อมโยงเหตุการณ์กับกรอบ MITRE ATT&CK [21] แนวทางเหล่านี้ยืนยันว่า Sysmon เป็นแหล่งข้อมูลพฤติกรรมบนเครื่องที่เหมาะสมต่อการเรียนรู้ของเครื่อง

### 2.2 ชุดข้อมูลเชิงพลวัตและการวิเคราะห์เชิงพฤติกรรม

Sgandurra และคณะ [6] เสนอ EldeRan สำหรับจำแนก Ransomware โดยใช้ Cuckoo Sandbox เก็บพฤติกรรมขณะทำงาน เช่น การเรียก Windows API และการดำเนินการกับ Registry ไฟล์ และไดเรกทอรี งาน [7] สร้างชุดข้อมูลคุณลักษณะเชิงพลวัตสำหรับตรวจจับ Ransomware และเปรียบเทียบแบบจำลองหลายชนิด ส่วนงาน [8] แสดงการใช้การเรียนรู้เชิงลึกกับข้อมูลมัลแวร์ บทความนี้นำแนวคิด Host Behavioral Features มาประยุกต์กับ Sysmon Event Logs โดยตรง แทนการใช้ Cuckoo Sandbox เพื่อให้ได้ข้อมูลระดับกระบวนการที่ละเอียดและใกล้เคียงการใช้งานจริง

### 2.3 การวิเคราะห์ระดับกระบวนการและข้ามแพลตฟอร์ม

งาน [9] แสดงว่าการวิเคราะห์ในระดับกระบวนการ (process-level) ให้ผลดีกว่าระดับเครื่อง (machine-level) โดยได้ค่า F1 ที่ 0.87 พร้อมอัตราการแข่งขันติดต่ำ แม้มีโปรแกรมพื้นหลังทำงานพร้อมกัน ด้านความแตกต่างข้ามระบบปฏิบัติการ งาน BarongTrace [4] สร้างชุดข้อมูลมัลแวร์บน Linux ด้วย Sysmon for Linux และพบว่าเมื่อใช้แบบจำลองที่ฝึกจากข้อมูล Windows

ไปทำนายข้อมูล Linux ค่า F1 ลดลงได้ถึง 0.54 ซึ่งยืนยันว่าข้อมูลจากสองระบบปฏิบัติการมีลักษณะต่างกัน และเป็นเหตุผลที่บทความนี้ออกแบบคุณลักษณะเชิงพฤติกรรมกลางระดับกระบวนการ

## 2.4

### อคติในการประเมินและการรั่วไหลของข้อมูลในความมั่นคงปลอดภัยเชิงเรียนรู้

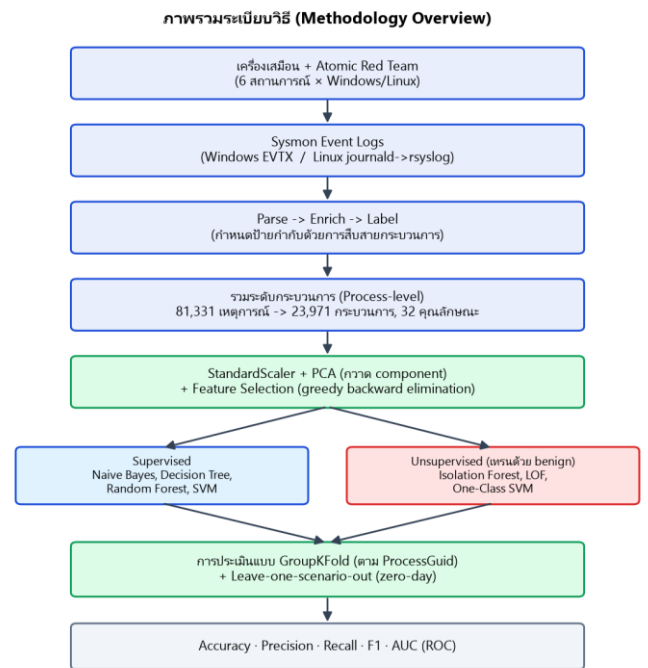
Arp และคณะ [2] ทบทวนงานด้านความมั่นคงปลอดภัยเชิงเรียนรู้ 30 บทความ และชี้ให้เห็นกับดักที่พบบ่อย เช่น การรั่วไหลของข้อมูล (data snooping) การเลือกตัวอย่างที่ไม่เป็นตัวแทน และการวัดผลที่ทำให้ค่าดูสูงเกินจริง ส่วน Pendlebury และคณะ [3] เสนอกรอบ TESSERACT เพื่อขจัดอคติเชิงพื้นที่และเชิงเวลาในการจำแนกมัลแวร์ บทความนี้นำข้อเสนอแนะดังกล่าวมาปฏิบัติโดยตรง ด้วยการรายงานผลภายใต้การแบ่งข้อมูลแบบจัดกลุ่มตามกระบวนการ (GroupKFold) และการประเมินแบบกันสถานการณ์ออก (leave-one-scenario-out) เพื่อจำลองการเผชิญภัยที่ไม่เคยพบ

## 2.5 การตรวจจับด้วยข้อมูลสืบสาย (Provenance)

แนวคิดการใช้ความสัมพันธ์เชิงสืบสายของกระบวนการเพื่อตรวจจับภัยคุกคามขั้นสูงมีงานรองรับ เช่น UNICORN [10] ที่ใช้กราฟสืบสาย (provenance graph) ตรวจจับ Advanced Persistent Threat แบบไม่ต้องมีลายเซ็น และ Kairos [11] ที่ตรวจจับและสืบสวนการบุกรุกจากประวัติการทำงานทั้งระบบ บทความนี้นำหลักการสืบสายกระบวนการมาใช้ในขั้นการกำหนดป้ายกำกับ โดยระบุว่าเหตุการณ์ใดสืบสายมาจากกระบวนการที่ถูกปล่อยในโดเมนทริททดสอบ

## 3. ระเบียบวิธีวิจัย

ระเบียบวิธีของบทความนี้ประกอบด้วย การเก็บบันทึกเหตุการณ์จากการจำลองพฤติกรรม การแปลงเป็นคุณลักษณะระดับกระบวนการ การลดมิติและเลือกคุณลักษณะ และการฝึกแบบจำลองทั้งแบบมีผู้สอนและไม่มีผู้สอน ภาพรวมทั้งกระบวนการแสดงในภาพ 1



ภาพ 1 ภาพรวมระเบียบวิธี (Methodology Overview)

## 3.1 สภาพแวดล้อมการทดลอง

การทดลองดำเนินบนเครื่องโฮสต์ Windows 11 และใช้ VMware Workstation ร่วมกับ Vagrant ในการจัดการเครื่องเสมือน เครื่องเป้าหมายฝั่ง Linux ใช้ Ubuntu 22.04 ติดตั้ง Sysmon for Linux (schema 4.81) ส่งบันทึกผ่าน journald → rsyslog → TCP ไปยังตัวรับบนโฮสต์ ส่วนเครื่องเป้าหมายฝั่ง Windows ติดตั้ง Sysmon และส่งออกบันทึกในรูปแบบ EVT X เนื่องจากทรัพยากรหน่วยความจำจำกัด จึงรันเครื่องเสมือนทีละเครื่องแบบลำดับ (sequential) และคืนสภาพ (revert snapshot) ก่อนเริ่มทุกสถานการณ์เพื่อความสามารถในการทำซ้ำ ทั้งพฤติกรรมปกติและพฤติกรรมอันตรายถูกเก็บบนเครื่องเดียวกันเพื่อป้องกันการรั่วไหลระดับเครื่อง (VM leakage) [2]

## 3.2 การจำลองพฤติกรรมและสถานการณ์

ใช้ Atomic Red Team [22] จำลองเทคนิคตามกรอบ MITRE ATT&CK [21] โดยรันชุดทดสอบบนเครื่องเป้าหมายเอง

และใช้เซิร์ฟเวอร์จำลอง Command-and-Control และ Mining Pool ภายในแล็บสำหรับพฤติกรรมเชิงเครือข่าย แบ่งเป็น 6 ประเภทสถานการณ์ต่อแพลตฟอร์ม (ตาราง 1) การเลือกมัลแวร์ห้าตระกูลนี้จึงใจให้ครอบคลุมพฤติกรรมโฮสต์ที่แตกต่างกัน ได้แก่ การกระทำต่อไฟล์จำนวนมาก (ransomware) การคงอยู่และแก้ไข registry (trojan) การติดต่อเครือข่ายซ้ำ (botnet) การใช้ทรัพยากรต่อเนื่องพร้อมต่อ pool (miner) และการยกระดับสิทธิ์/แทรกโค้ด (exploit) ทำให้ชุดข้อมูลไม่ลำเอียงไปยังเหตุการณ์ชนิดใดชนิดหนึ่ง

ตาราง 1 ประเภทสถานการณ์ต่อแพลตฟอร์ม

| สถานการณ์    | ป้ายกำกับ | ตัวอย่างเทคนิค ATT&CK              |
|--------------|-----------|------------------------------------|
| Benign       | 0         | กิจกรรมผู้ใช้ปกติ                  |
| Ransomware   | 1         | T1486, T1083, T1005, T1070.004     |
| Trojan       | 1         | T1027, T1036.003, T1547.001, T1112 |
| Botnet       | 1         | T1071.001, T1132.001, T1105        |
| Cryptominer  | 1         | T1496, T1053, T1057                |
| Exploitation | 1         | T1055, T1548.002, T1134, T1218.011 |

### 3.3 ท่อประมวลผลข้อมูล

บันทึกดิบถูกแปลงผ่านขั้นตอน parse → enrich → label ได้แก่ (1) แยกวิเคราะห์ XML เป็นตาราง (2) เติมข้อมูลระดับกระบวนการ เช่น บรรทัดคำสั่ง ค่าแฮช และกระบวนการแม่ และ (3) กำหนดป้ายกำกับ จากนั้นรวมทุกรอบทดลองเป็นชุดข้อมูลเดียวโดยคงคอลัมน์ระบุ แพลตฟอร์ม เครื่อง และรอบทดลองไว้เพื่อการตรวจสอบย้อนกลับ

### 3.4 การกำหนดป้ายกำกับด้วยการสืบสายกระบวนการ

ป้ายกำกับ (label = 1 คืออันตราย) กำหนดโดยการสืบสายกระบวนการ (process lineage): เหตุการณ์ใดที่สืบสายมาจากกระบวนการที่ถูกปล่อยจากไดเรกทอรีทดสอบจะถูกกำกับเป็นอันตราย

แนวทางนี้สอดคล้องกับการตรวจจับด้วยข้อมูลสืบสาย [10], [11] และให้ผลแม่นยำกว่าการกำหนดตามช่วงเวลา (session labeling) ซึ่งเคยวัดพบว่ากำหนดป้ายผิดสูง อย่างไรก็ตาม การมีป้ายกำกับว่าอันตรายไม่ได้หมายความว่าทุกเหตุการณ์เป็นพฤติกรรมมัลแวร์ในตัวเอง

จึงจำแนกความแรงของหลักฐานออกเป็นระดับ

ตั้งแต่หลักฐานตรงในคอลัมน์ของเหตุการณ์เอง

ไปจนถึงการติดป้ายเพราะสายเลือดล้วน

### 3.5 การสร้างคุณลักษณะเชิงพฤติกรรม

แต่ละกระบวนการ (จัดกลุ่มด้วย ProcessGuid ร่วมกับรหัสเครื่องและรอบทดลอง)

ถูกแปลงเป็นเวกเตอร์คุณลักษณะเชิงตัวเลข 32 ตัว แบ่งเป็น 5 กลุ่มดังตาราง 2

การรวมเหตุการณ์เป็นสถิติระดับกระบวนการช่วยให้แบบจำลองวิเคราะห์พฤติกรรมจากหลายเหตุการณ์ร่วมกันแทนการพิจารณาเหตุการณ์เดี่ยว ทั้งนี้ต้องแยกกรณี “ไม่มีพฤติกรรมเกิดขึ้น” ออกจากกรณี “แหล่งข้อมูลไม่รองรับการเก็บพฤติกรรมนั้น” เนื่องจาก Windows และ Linux เก็บเหตุการณ์ได้ไม่เท่ากัน

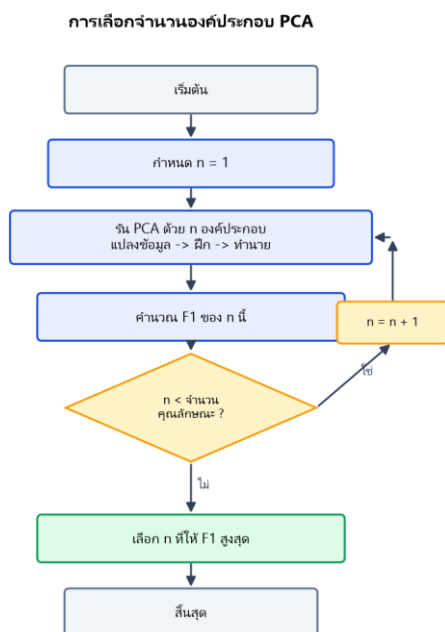
ตาราง 2 กลุ่มคุณลักษณะเชิงพฤติกรรมระดับกระบวนการ

| กลุ่ม             | ตัวอย่างคุณลักษณะ                                     |
|-------------------|-------------------------------------------------------|
| วงจรกระบวนการ     | จำนวน ProcessCreate/Terminate, จำนวน child, ระยะเวลา  |
| กิจกรรมไฟล์       | จำนวน FileCreate/FileDelete, จำนวนพาทที่แตกต่างกัน    |
| กิจกรรมเครือข่าย  | จำนวน NetworkConnect, จำนวน IP/พอร์ตปลายทางที่ต่างกัน |
| กิจกรรม registry  | จำนวน RegistrySetValue/RegistryAddDelete              |
| คำสั่งและเมทาดาตา | ความยาวคำสั่ง, ระดับ integrity, ข้อมูลกระบวนการแม่    |

### 3.6 การลดมิติด้วย PCA และการเลือกคุณลักษณะ

นำคุณลักษณะมาปรับมาตรฐานด้วย StandardScaler ก่อนศึกษาการลดมิติด้วย Principal Component Analysis (PCA) โดยกวาดจำนวนองค์ประกอบตั้งแต่ 1 ถึงจำนวนคุณลักษณะเต็ม แล้วเลือกค่าที่ให้ F1 สูงสุดบนชุด validation เพื่อป้องกันการรั่วไหลของข้อมูล พารามิเตอร์ของ

StandardScaler และ PCA ถูกเรียนรู้จากชุด train เท่านั้น แล้วนำการแปลงเดียวกันไปใช้กับ validation และ test [2] นอกจากนี้ยังศึกษาการเลือกคุณลักษณะแบบตัดถอยหลังอย่างละโมบ (greedy backward elimination) เพื่อระบุว่าคุณลักษณะใดแบกสัญญาณจริง และเพื่อคัดคุณลักษณะที่สืบเนื่องจากกระบวนการกำหนดป้ายกำกับ (derived features) ออก ผลของทั้ง PCA และการเลือกคุณลักษณะรายงานในหัวข้อ 4.5 การจัดการความไม่สมดุลของคลาสพิจารณาใช้เทคนิค SMOTE [18] เฉพาะบนชุด train ขั้นตอนการกวาดจำนวนองค์ประกอบ PCA (ทำตามงานอ้างอิง [1]) แสดงเป็นผังงานในภาพ 2



ภาพ 2 ผังงานการเลือกจำนวนองค์ประกอบ PCA (ตามแนวทางงานอ้างอิง [1])

### 3.7 โปรโตคอลการประเมิน

เนื่องจากเหตุการณ์หลายแฉมาจากกระบวนการเดียวกัน การแบ่งฝึก/ทดสอบแบบสุ่มรายแฉจะทำให้เหตุการณ์จากกระบวนการเดียวกันปรากฏทั้งในชุดฝึกและชุดทดสอบ ส่งผลให้ค่าที่รายงานสูงเกินจริง [2], [3]

บทความนี้จึงรายงานผลหลักด้วย GroupKFold แบ่งตาม ProcessGuid และประเมินระดับกระบวนการ พร้อมตัดคุณลักษณะที่สืบเนื่องจากกระบวนการกำหนดป้ายกำกับ ออก

และประเมินความสามารถต่อภัยที่ไม่เคยพบด้วยการกันสถานการณ์ออกทีละชนิด (leave-one-scenario-out) วัดผลด้วย Accuracy, Precision, Recall, F1 และ AUC

## 4. การทดลองและผลลัพธ์

### 4.1 องค์ประกอบชุดข้อมูล

ชุดข้อมูลรวมมี 81,331 เหตุการณ์ 80 คอลัมน์ เมื่อรวมระดับกระบวนการได้ 23,971 ตัวอย่าง องค์ประกอบตามตาราง 3 จุดเด่นคือสัดส่วนตัวอย่างอันตรายของสองแพลตฟอร์มใกล้เคียงกันมาก (Linux 34.2% เทียบ Windows 33.3%) จึงลดความเสี่ยงที่แพลตฟอร์มจะกลายเป็นตัวแทนของป้ายกำกับ

ตาราง 3 องค์ประกอบตามแพลตฟอร์มและป้ายกำกับ

| แพลตฟอร์ม | ปกติ   | อันตราย | รวม    | %อันตราย |
|-----------|--------|---------|--------|----------|
| Linux     | 29,805 | 15,506  | 45,311 | 34.2%    |
| Windows   | 24,027 | 11,993  | 36,020 | 33.3%    |
| รวม       | 53,832 | 27,499  | 81,331 | 33.8%    |

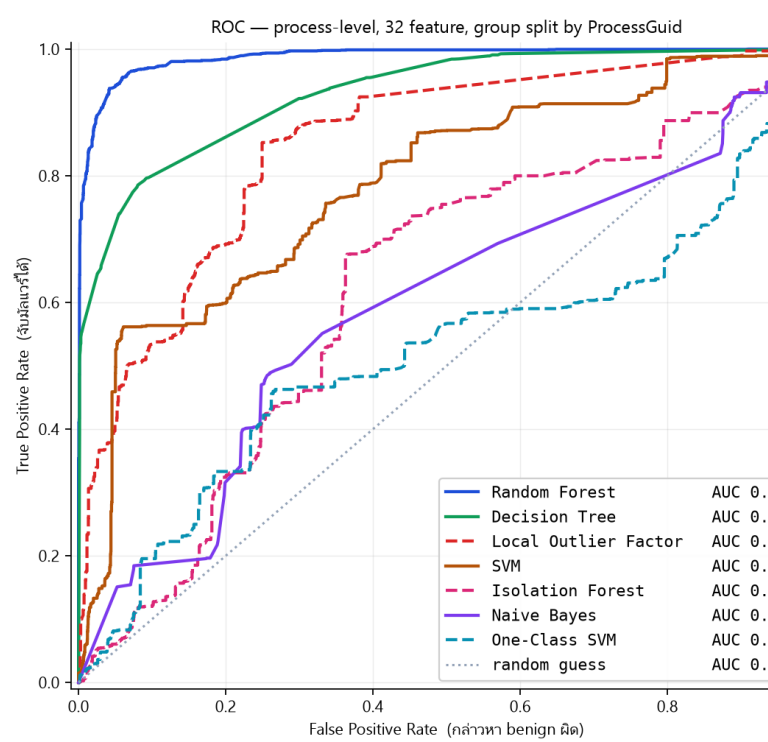
### 4.2 ผลการจำแนกระดับกระบวนการ

ตาราง 4 แสดงผลภายใต้โปรโตคอล GroupKFold ระดับกระบวนการ (32 คุณลักษณะที่ควบคุมการรั่วไหล) แบบจำลอง Random Forest [12] ให้ผลดีที่สุด ความแม่นยำ 94.9% ค่า F1 0.924 และ AUC 0.988 สำหรับแบบไม่มีผู้สอน (ฝึกด้วยข้อมูลปกติเท่านั้น) Local Outlier Factor [14] ให้ผลดีที่สุดสอดคล้องกับข้อสรุปของงานอ้างอิง [1] โดยได้ AUC 0.849 สูงกว่าเส้นฐานแจ้งเตือนทุกแฉ (F1 0.500) อย่างชัดเจน ภาพ 3 เปรียบเทียบค่า Precision, Recall และ F1 ของแบบจำลองทั้งเจ็ด และภาพ 4 แสดงเส้นโค้ง ROC

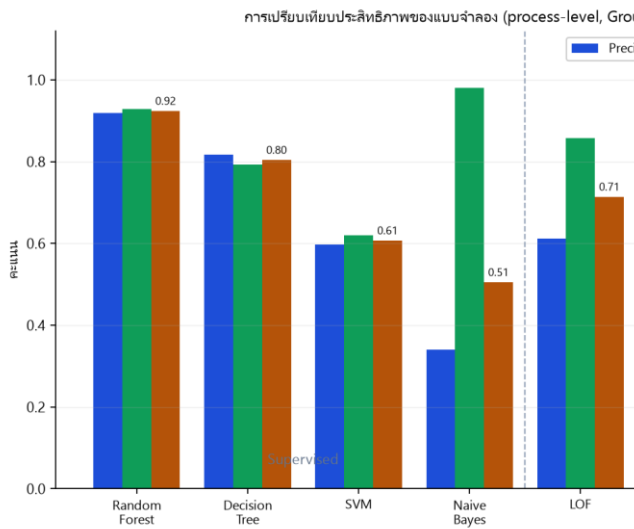
ตาราง 4 ผลการจำแนกระดับกระบวนการ (GroupKFold, 32 คุณลักษณะ)

| ชนิด     | แบบจำลอง      | Acc   | Prec  | Rec   | F1    | AUC   |
|----------|---------------|-------|-------|-------|-------|-------|
| มีผู้สอน | Random Forest | 0.949 | 0.919 | 0.929 | 0.924 | 0.988 |

|             |                      |      |      |      |      |      |
|-------------|----------------------|------|------|------|------|------|
| มีผู้สอน    | Decision Tree        | 0.87 | 0.81 | 0.79 | 0.80 | 0.93 |
|             | Tree                 | 1    | 7    | 2    | 4    | 0    |
| มีผู้สอน    | SVM                  | 0.73 | 0.59 | 0.61 | 0.60 | 0.78 |
|             | SVM                  | 3    | 7    | 9    | 7    | 6    |
| มีผู้สอน    | Naive Bayes          | 0.35 | 0.34 | 0.98 | 0.50 | 0.59 |
|             | Naive Bayes          | 9    | 0    | 0    | 5    | 4    |
| ไม่มีผู้สอน | Local Outlier Factor | 0.77 | 0.61 | 0.85 | 0.71 | 0.84 |
|             | Local Outlier Factor | 1    | 2    | 7    | 4    | 9    |
| ไม่มีผู้สอน | Isolation Forest     | 0.64 | 0.47 | 0.66 | 0.55 | 0.62 |
|             | Isolation Forest     | 7    | 9    | 7    | 8    | 2    |
| ไม่มีผู้สอน | One-Class SVM        | 0.55 | 0.37 | 0.51 | 0.43 | 0.49 |
|             | One-Class SVM        | 4    | 6    | 3    | 4    | 9    |
| เส้นฐาน     | แจ้งเตือนทุกแถว      | 0.33 | 0.33 | 1.00 | 0.50 | -    |
|             | แจ้งเตือนทุกแถว      | 4    | 4    | 0    | 0    |      |



ภาพ 4 เส้นโค้ง ROC ของแบบจำลองทั้งเจ็ด (ระดับกระบวนการ, group split ตาม ProcessGuid)



ภาพ 3 การเปรียบเทียบประสิทธิภาพของแบบจำลอง (Precision, Recall, F1)

#### 4.3 การประเมินแบบ zero-day (leave-one-scenario-out)

เพื่อประเมินความสามารถในการตรวจจับการโจมตีชนิดที่ไม่เคยพบในการฝึก ตามแนวคิดการขจัดอคติของ [3] เราใช้การกั้นสถานการณ์ออกทีละชนิดจนครบ ผลในตาราง 5 แสดงว่า Random Forest ชนะเส้นฐาน 8 จาก 10 สถานการณ์ และ Local Outlier Factor ชนะเส้นฐานครบ 10 จาก 10 สถานการณ์

ยืนยันว่าคุณลักษณะที่ใช้อย่างได้กับพฤติกรรมที่ไม่เคยพบมาก่อนในระดับหนึ่ง และการรวมสองแพลตฟอร์มให้ผลดีกว่าการใช้ Linux เพียงอย่างเดียว

ตาราง 5 สรุปผล zero-day (ค่าเฉลี่ยข้าม 10 สถานการณ์)

| แบบจำลอง             | F1 เฉลี่ย | AUC เฉลี่ย | ชนะเส้นฐาน |
|----------------------|-----------|------------|------------|
| Random Forest        | 0.764     | 0.897      | 8/10       |
| Local Outlier Factor | 0.728     | 0.780      | 10/10      |
| Decision Tree        | 0.688     | 0.820      | 7/10       |

4.4 การวิเคราะห์การรั่วไหลของข้อมูล (Data Leakage)

การรั่วไหลของข้อมูลเป็นประเด็นสำคัญที่สุดของบทความนี้ เนื่องจากทำให้ค่าที่รายงานดูดีเกินจริงโดยไม่ได้สะท้อนความสามารถจริงในการตรวจจับ [2], [3] เราจึงตรวจสอบแหล่งการรั่วไหลอย่างเป็นระบบสี่ประเภท (1) การรั่วไหลจากโปรโตคอลการประเมิน เมื่อเหตุการณ์จากกระบวนการเดียวกันปรากฏทั้งในชุดฝึกและชุดทดสอบ (2) การรั่วไหลระดับแพลตฟอร์ม จากรูปแบบคำว่างของคอลัมน์ที่ต่างกันระหว่าง Windows และ Linux ทำให้แบบจำลองแยกแพลตฟอร์มได้เกือบสมบูรณ์ (ราว 100%) หากไม่ควบคุม หลังตัดคอลัมน์ที่รั่วออกความสามารถในการแยกแพลตฟอร์มลดลงเข้าใกล้ระดับการเดาสุ่ม (3) การรั่วไหลจากขั้นเติมข้อมูล (enrichment) เมื่อกระบวนการที่เกิดขึ้นหลัง Sysmon มีบรรทัดคำสั่งแต่กระบวนการที่เกิดขึ้นก่อนไม่มี ทำให้ “บรรทัดคำสั่งว่าง” กลายเป็นตัวชี้ป้ายกำกับโดยบังเอิญ และ (4) การรั่วไหลจากคุณลักษณะที่สืบเนื่องจากการกำหนดป้ายกำกับ เช่น ไดรเวอร์ทำงาน (ซึ่งตรงกับไดเรกทอรีทดสอบที่ใช้ตัดสินป้ายกำกับ) และความลึกของสายกระบวนการ แนวทางนี้สอดคล้องกับงานอ้างอิง [1] ที่ตัดคอลัมน์ Image, ProcessGUID, ProcessId, timestamp และ host\_name ออกด้วยเหตุผลเดียวกัน คือคอลัมน์เหล่านี้สัมพันธ์โดยตรงกับตัวแปรเป้าหมาย

ตาราง 6 แสดงความอ่อนไหวของค่าที่รายงานต่อโปรโตคอลการประเมินบนแบบจำลองและข้อมูลชุดเดียวกัน การแบ่งแบบสุ่มรายแถวให้ F1 สูงถึง 0.961 แต่เมื่อเปลี่ยนเป็นการแบ่งแบบจัดกลุ่มตามกระบวนการ ค่าลดลงเหลือ 0.811 และเมื่อประเมินแบบกันสถานการณ์ออก (zero-day) ลดลงเหลือ 0.634 ความต่างกว่า 30 จุดนี้เกิดจากข้อมูลชุดเดียวกันทุกแถว ต่างกันเพียงวิธีแบ่ง จึงยืนยันว่าการรายงานตัวเลขต้องระบุโปรโตคอลเสมอ

ตาราง 6 ความอ่อนไหวต่อโปรโตคอล (Random Forest, ข้อมูลชุดเดียวกัน)

| วิธีแบ่ง train/test | Accuracy | F1 |
|---------------------|----------|----|
|---------------------|----------|----|

|                                   |       |       |
|-----------------------------------|-------|-------|
| สุ่มรายแถว (train_test_split)     | 0.974 | 0.961 |
| GroupKFold ตาม ProcessGuid        | 0.881 | 0.811 |
| Leave-one-scenario-out (zero-day) | 0.636 | 0.634 |

4.5 ผลของการเลือกคุณลักษณะและ PCA

การเลือกคุณลักษณะแบบตัดโดยหลังพบว่าคุณลักษณะเพียง 4-5 ตัวแบกสัญญาณส่วนใหญ่ อย่างไรก็ตาม คุณลักษณะเหล่านี้ส่วนมากไม่ใช่ “พฤติกรรม” แต่เป็นการรั่วไหลของท่อข้อมูล เช่น ไดรเวอร์ทำงานและความลึกของสายกระบวนการ ตาราง 7 แสดงว่าการตัดคุณลักษณะที่รั่ว (derived) ออกทำให้ F1 ลดลงเล็กน้อย (0.9475 → 0.9241) แต่เมื่อตัดคุณลักษณะความยาวคำสั่งออกด้วยจนเหลือเฉพาะพฤติกรรมล้วน F1 ลดลงมากถึง 0.610 ซึ่งชี้ว่าคะแนนส่วนหนึ่งมาจากความยาวคำสั่งของชุดทดสอบอัตโนมัติ ไม่ใช่พฤติกรรมที่ Sysmon เก็บ ข้อสังเกตนี้สอดคล้องกับงานอ้างอิง [1] ที่ตัดคอลัมน์ Image ทั้งเพราะสัมพันธ์โดยตรงกับป้ายกำกับ บทความนี้จึงรายงานผลด้วยชุดคุณลักษณะที่ควบคุมการรั่วไหลแล้วเป็นหลัก และรายงานคู่กับชุดที่ไม่ควบคุมเพื่อความโปร่งใส

ตาราง 7 ความอ่อนไหวต่อชุดคุณลักษณะ (ระดับกระบวนการ)

| ชุดคุณลักษณะ                    | จำนวน | RF F1 | LOF AUC |
|---------------------------------|-------|-------|---------|
| ทั้งหมด                         | 35    | 0.948 | 0.866   |
| ตัด derived (ควบคุมการรั่วไหล)  | 32    | 0.924 | 0.849   |
| พฤติกรรมล้วน (ตัดความยาวคำสั่ง) | 30    | 0.610 | -       |

สำหรับการลดมิติ การกวาดจำนวนองค์ประกอบ PCA พบว่าจำนวนที่ให้ผลดีที่สุดเกือบเท่ากับจำนวนคุณลักษณะเต็ม และที่ระดับเหตุการณ์ PCA กลับทำให้ F1 ลดลงเล็กน้อย (0.951 → 0.936) เนื่องจากคุณลักษณะผ่านการเข้ารหัสแบบ label encoding และความยาวสตริงมาแล้ว จึงไม่มีความสัมพันธ์เชิงเส้นให้ PCA รวบรวมได้นัก

ผลนี้บ่งชี้ว่าการควบคุมการรั่วไหลและการเลือกคุณลักษณะมีผลต่อความน่าเชื่อถือมากกว่าการลดมิติด้วย PCA สำหรับชุดข้อมูลนี้

4.6 การเปรียบเทียบองค์ประกอบกับงานอ้างอิง

ตาราง 8 เปรียบเทียบสัดส่วนชนิดเหตุการณ์ระหว่างชุดข้อมูลของบทความนี้กับชุด NLME ของงานอ้างอิง [1] จะเห็นว่าชุดข้อมูลนี้ครอบคลุมเหตุการณ์เชิงเครือข่าย (NetworkConnect) การลบไฟล์ และการอ่านดิสก์ระดับต่ำที่งานอ้างอิงแทบไม่มี ทำให้เหมาะกับการวิเคราะห์พฤติกรรม C2/beaconing และ ransomware มากขึ้น

ตาราง 8 สัดส่วนชนิดเหตุการณ์ (%): บทความนี้ เทียบ NLME [1]

| ชนิดเหตุการณ์        | บทความนี้ | NLME [1] |
|----------------------|-----------|----------|
| NetworkConnect (3)   | 2.8       | 0.03     |
| FileDelete (23)      | 9.6       | 0        |
| RawAccessRead (9)    | 6.1       | 0        |
| ProcessTerminate (5) | 33.0      | 2.1      |
| ProcessCreate (1)    | 28.7      | 29.5     |
| FileCreate (11)      | 11.7      | 39.0     |

5. อภิปรายผล

ผลการทดลองสอดคล้องกับงานอ้างอิงในสองประเด็น คือ Random Forest ให้ผลดีที่สุดในกลุ่มมีผู้สอน และ Local Outlier Factor ดีที่สุดในกลุ่มไม่มีผู้สอน [1], [14] นอกจากนี้การจำแนกระดับกระบวนการให้ผลดีกว่าระดับเหตุการณ์เดี่ยวอย่างชัดเจน สอดคล้องกับ [9] เนื่องจากเหตุการณ์เดี่ยวของมัลแวร์มักคล้ายพฤติกรรมปกติ สิ่งที่น่าสังเกตได้จริงคือรูปแบบรวมของกระบวนการ

ประเด็นที่บทความนี้เน้นเป็นพิเศษคือความน่าเชื่อถือของการวัดผล เราแสดงเชิงประจักษ์ว่าค่าที่รายงานขึ้นกับ (1) โปรโตคอลการประเมิน (ตาราง 6) และ (2) ชุดคุณลักษณะที่เลือก (ตาราง 7) มากกว่าปริมาณข้อมูล ตามที่ [2], [3] เตือนไว้ การควบคุมการรั่วไหลของข้อมูลจึงสำคัญกว่าการเพิ่มจำนวนเหตุการณ์ และการลดมิติด้วย PCA ให้ประโยชน์จำกัดกับชุดข้อมูลนี้เพราะคุณลักษณะผ่านการเข้ารหัสมาแล้ว

ให้สมแล้ว การรายงานตัวเลขจึงควรระบุทั้งโปรโตคอลและชุดคุณลักษณะเสมอ ข้อจำกัดของงานได้แก่ จำนวนเครื่องที่ใช้เก็บข้อมูลยังน้อย ความหลากหลายของพฤติกรรมปกติยังจำกัด (ทำให้คุณลักษณะความยาวคำสั่งแยกคลาสได้ง่ายเกินจริง) และการจำลองด้วย Atomic Red Team อาจให้พฤติกรรมน้อยกว่ามัลแวร์จริงในบางเหตุการณ์

6. สรุปและงานในอนาคต

บทความนี้นำเสนอชุดข้อมูล Sysmon แบบสองแพลตฟอร์มจำนวน 81,331 เหตุการณ์สำหรับการตรวจจับมัลแวร์ พร้อมวิเคราะห์การรั่วไหลของข้อมูลอย่างเป็นระบบ ภายใต้โปรโตคอลที่ป้องกันการรั่วไหล แบบจำลอง Random Forest ระดับกระบวนการให้ความแม่นยำ 94.9% และ F1 0.924 งานในอนาคตได้แก่ การเพิ่มความหลากหลายของพฤติกรรมปกติ การเพิ่มจำนวนเครื่องและรอบทดลองเพื่อเสริมความแข็งแกร่งของการประเมินแบบ zero-day และการหลอมรวมสัญญาณระดับเครือข่าย เข้ากับบันทึกโฮสต์เพื่อยกระดับการตรวจจับพฤติกรรม C2

เอกสารอ้างอิง

[1] R. M. R. Achmad, D. P. Nariswari, B. A. Pratomo, and H. Studiawan, "Sysmon event logs for machine learning-based malware detection," Cyber Security and Applications, vol. 3, 100110, 2025.

[2] D. Arp et al., "Dos and Don'ts of Machine Learning in Computer Security," in Proc. 31st USENIX Security Symposium, 2022, pp. 3971-3988.

[3] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time," in Proc. 28th USENIX Security Symposium, 2019, pp. 729-746.

[4] B. A. Pratomo, S. A. Kosim, H. Studiawan, and A. O. Prabowo, "BarongTrace: A Malware Event Log



- Dataset for Linux," in *Advanced Information Networking and Applications*, LNDECT, vol. 202, pp. 48-60, 2024.
- [5] R. V. Mahmoud, M. Anagnostopoulos, S. Pastrana, and J. M. Pedersen, "Redefining Malware Sandboxing: Enhancing Analysis Through Sysmon and ELK Integration," *IEEE Access*, vol. 12, pp. 68624-68636, 2024.
- [6] D. Sgandurra, L. Munoz-Gonzalez, R. Mohsen, and E. C. Lupu, "Automated Dynamic Analysis of Ransomware: Benefits, Limitations and use for Detection," *arXiv:1609.03020*, 2016.
- [7] J. A. Herrera-Silva and M. Hernandez-Alvarez, "Dynamic Feature Dataset for Ransomware Detection Using Machine Learning Algorithms," *Sensors*, vol. 23, no. 3, 1053, 2023.
- [8] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, and S. Venkatraman, "Robust Intelligent Malware Detection Using Deep Learning," *IEEE Access*, vol. 7, pp. 46717-46738, 2019.
- [9] B. A. Pratomio et al., "Enhancing Enterprise Network Security: Comparing Machine-Level and Process-Level Analysis for Dynamic Malware Detection," *arXiv:2310.18165*, 2023.
- [10] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime Provenance-Based Detector for Advanced Persistent Threats," in *Proc. NDSS Symposium*, 2020.
- [11] Z. Cheng et al., "Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance," in *Proc. IEEE Symposium on Security and Privacy*, 2024.
- [12] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [13] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation Forest," in *Proc. IEEE ICDM*, 2008, pp. 413-422.
- [14] M. M. Breunig, H. P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying Density-Based Local Outliers," in *Proc. ACM SIGMOD*, 2000, pp. 93-104.
- [15] B. Scholkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the Support of a High-Dimensional Distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443-1471, 2001.
- [16] C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, no. 3, pp. 273-297, 1995.
- [17] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1-58, 2009.
- [18] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.
- [19] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [20] M. Russinovich and T. Garnier, "Sysmon (System Monitor)," *Microsoft Sysinternals*, 2024. [Online]. Available: <https://learn.microsoft.com/sysinternals/downloads/sysmon>
- [21] B. E. Strom et al., "MITRE ATT&CK: Design and Philosophy," *The MITRE Corporation*, 2020.
- [22] Red Canary, "Atomic Red Team," *GitHub repository*, 2024. [Online]. Available: <https://github.com/redcanaryco/atomic-red-team>